



兰州工业学院

实 验 报 告

(2023 —2024 学年第 1 学期)

课程名称 面向对象程序设计

专业班级 专升本电信 23

学 号 ZB2305010116

姓 名 姜 勇

实验项目	实验八 SQL 数据库的操作		
实验时间	2023 年 12 月 20 日	实验地点	信息与通信工程专业实验室 (DSP)

一、实验目的

1. 熟悉是 Qt 与数据库的连接和配置;
2. 理解 Qt 配合数据库完成数据信息的增删改查。

二、实验环境

硬件：计算机一台；

软件：VC++ 6.0 或 Code::Blocks。

三、实验内容

实现一个信息查询窗口，能够完成对数据库的对接和相关操作。具体要求如下：

- (1) 主窗口标题名称为学号后两位+姓名。
- (2) 整体包含信息显示、信息检索、信息添加等模块。

(3) 对接数据库后，新建数据表，并在数据表中设置相关的列向量（至少 4 项），包括但不限于：学号 ID、姓名 name、性别 gender、年龄 age、分数 score 等，对应的变量属性可以自由定义。

```
void initDatabase(QWidget *parent){
    db.setHostName("192.168.1.10");
    db.setUserName("root");
    db.setPassword("123456");
    db.setDatabaseName("students");
    if(!db.open()){
        QMessageBox::warning(parent,"打不开",db.lastError().text());
        return;
    }

    query.exec("create table if not exists "+tableName+"(id int primary key auto_increment, "
        +name+" varchar(20), "
        +age+" int, "
        +score+" int);");
}
}
```

(3) 信息显示窗口能够显示数据表内的全部内容，并铺满整个窗口。

id	name	age	score
1	aba	11	11
4	yangjiangbo	22	10

(4) 信息检索和信息添加完成对数据条目的增删改查，可以使用同一个检索属性，也可以使用不同的检索属性。

ID	Name	Age	Score
查询	增加	修改	删除

(5) 自由发挥：完成数据库的操作与菜单栏的关联。

mainwindow.h 文件

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H
#include <QMainWindow>
#include<QSqlDatabase>
#include<QSqlQuery>
#include<QSqlError>
#include<QSqlTableModel>
#include<QDebug>
#include<QMessageBox>
#include <QDesktopServices>
QT_BEGIN_NAMESPACE
namespace Ui { class MainWindow; }
```

```

QT_END_NAMESPACE

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

    void connect_open();
    void lineEditclear();
    void databaseclose();

private slots:
    void on_pushButton_5_clicked();
    void on_pushButton_clicked();
    void on_pushButton_2_clicked();
    void on_pushButton_3_clicked();
    void on_pushButton_4_clicked();
    void on_pushButton_6_clicked();
    void on_actiona_triggered();
    void on_actionV_triggered();
    void on_actionC_triggered();
    void on_actionxia_triggered();

private:

```

```

    Ui::MainWindow *ui;

    QSqlDatabase db;

    QSqlTableModel *model;

    QSqlQuery query;
};

#endif // MAINWINDOW_H

mainwindow.cpp 文件
#include "mainwindow.h"
#include "ui_mainwindow.h"

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);

    connect_open(); // 调用 connect_open() 函数来建立数据库连接

    model=new QSqlTableModel(this); // 创建一个新的
    QSqlTableModel 对象，并将其赋值给变量 model
    model->setEditStrategy(QSqlTableModel::OnManualSubmit); // 设置编辑策略为手动提交模式

    ui->tableView->setModel(model); // 将表格视图
    的模型设置为刚刚创建的 model

```

```

}
MainWindow::~MainWindow()
{
    delete ui;
}
void MainWindow::connect_open()//连接数据库
{
    db=QSqlDatabase::addDatabase("QODBC");
    db.setHostName("localhost");
    db.setDatabaseName("QT_book");
    db.setUserName("JY");
    db.setPassword("123456");
    if(db.open())
    {
        qDebug()<<"数据库连接成功";
        QSqlQuery query;
        QString creatTableStr = "CREATE TABLE
students (" "id char(10) NULL, "
        "name char(50) NULL, "
        "sex char(2) NULL, "
        "age char(50) NULL, "
        "class char(50) NULL,"

```

```

        " score char(50) NULL);";
query.prepare(creatTableStr);
    if(!query.exec()){
        qDebug()<<"query
error : "<<query.lastError();
    }
    else{
        qDebug()<<"creat table success!";
    }
}
else
{
    qDebug()<<"数据库连接失败";
    return;
}
}

void MainWindow::on_pushButton_5_clicked()
{
    model->setTable("students");// 设置表格模型的
表名为 "students"
    model->select();// 执行查询操作，从数据库中选择
所有记录

```

```

}

void MainWindow::on_pushButton_clicked()//查询成员
{
    QString aname = ui->lineEdit_2->text();

    if (aname.isEmpty())
    {
        QMessageBox::warning(this, "警告", "学生姓名为空");
        return;
    }

    model->setFilter(QString("name='%1' ").arg(aname));

    if (!model->select())
    {
        QMessageBox::critical(this, "错误", "查询失败");
        return;
    }

    if (model->rowCount() <= 0)
        QMessageBox::warning(this, "警告", "用户名

```



```

不存在");
}

void MainWindow::on_pushButton_2_clicked() //增加成员
{
    QSqlQuery selectQuery;
    selectQuery.prepare("SELECT name FROM students
WHERE name = ?");

    QString username=ui->lineEdit_2->text();
    selectQuery.addBindValue(username);
    selectQuery.exec();

    if (selectQuery.next()) {
        QMessageBox::warning(this, "警告", "学生已
存在");
        lineEditclear();
        return;
    }
    else
    {
        QSqlQuery query;
        query.prepare("insert          into          students

```

```

values(?, ?, ?, ?, ?, ?)");

    QString id=ui->lineEdit->text();
    QString name=ui->lineEdit_2->text();
    QString sex=ui->lineEdit_5->text();
    QString age=ui->lineEdit_3->text();
    QString banji=ui->lineEdit_6->text();
    QString score=ui->lineEdit_4->text();
    if(id==" "||name == " "||sex == " " ||age ==
    " "||banji == " "||ui->lineEdit_4->text()=="") //插入
    信息的时候需要输入完整的信息

    {

        QMessageBox::information(this,"提示","请输
    入完整的信息");

        return;
    }

    if(sex!="男"&&sex!="女")

    {

        QMessageBox::information(this, "提示", "请
    输入有效的性别");

        return;
    }

    if(age<"18"||age>"25")

```

```

        {
            QMessageBox::information(this, "提示", "请输入合适的年龄");
            return;
        }
        else
        {
            query.addBindValue(id);
            query.addBindValue(name);
            query.addBindValue(sex);
            query.addBindValue(age);
            query.addBindValue(banji);
            query.addBindValue(score);
            lineEditclear();
            if(query.exec())
            {
                QMessageBox::information(this, "提示", "添加学生成功");
            }
            else
            {
                QMessageBox::warning(this, "警告", "添加

```

```

        学生失败: " + query.lastError().text());
    }

}

}

}

}

void MainWindow::lineEditclear()
{
    ui->lineEdit->clear();
    ui->lineEdit_2->clear();
    ui->lineEdit_3->clear();
    ui->lineEdit_4->clear();
    ui->lineEdit_5->clear();
    ui->lineEdit_6->clear();
}

void MainWindow::on_pushButton_3_clicked()//修改成
员信息
{
    model->database().transaction();// 开始一个数
据库事务

    if(model->submitAll())// 提交所有数据库操作。如
果所有操作都成功，则进入 if 分支。

    {

```

```

        model->database().commit(); // 提交事务，
将所有更改保存到数据库中

        QMessageBox::information(this, "提示", "修改
成功");
    }

    else
    {

        model->database().rollback(); // 如果提交操
作失败，回滚事务，撤销所有未保存的更改

        QMessageBox::warning(this, "      警      告
", model->lastError().text());

    }

    model->select();
}

void MainWindow::on_pushButton_4_clicked() // 删除成
员
{

    int
currow=ui->tableView->currentIndex().row(); // 获 取
当前选中行的行号（currow）。

    if (currow >= 0)
    {

```

```

        model->removeRow(currow); //从模型（model）中移
除当前选中的行。

        int ok=QMessageBox::warning(this, "警告", "确定删
除？ ", QMessageBox::Yes, QMessageBox::No);

        if(ok==QMessageBox::No)

            model->revertAll(); //撤销所有更改

        else

            model->submitAll(); //提交所有更改
    }

    else
    {

        QMessageBox::warning(this, "警告", "请选中一行
数据");

    }
}

void MainWindow::on_pushButton_6_clicked()
{

    ui->lineEdit->clear();

    ui->lineEdit_2->clear();

    ui->lineEdit_3->clear();

    ui->lineEdit_4->clear();

    ui->lineEdit_5->clear();
}

```

```

        ui->lineEdit_6->clear();
    }
void MainWindow::databaseclose()
{
    if(db.open())
    {
        db.close();
    }
}
void MainWindow::on_actiona_triggered()
{
    QMessageBox::StandardButton reply;
    reply = QMessageBox::question(this, "退出", "
确定要退出吗?",

QMessageBox::Yes|QMessageBox::No);
    if (reply == QMessageBox::Yes) {
        qApp->quit();
    }
}
void MainWindow::on_actionV_triggered()
{

```

```

        QMessageBox::information(this, "开发者信息", "作者: JY\n 时间: 2023/12/20");
    }

void MainWindow::on_actionC_triggered()
{
    QMessageBox::information(this, "软件说明", "数据库的前端页面操作");
}

void MainWindow::on_actionxia_triggered()
{
    QString projectDirectory =
"C:\\\\Users\\\\JY\\\\Desktop\\\\drill\\\\QT\\\\shiyang8"; // 替
换为你的项目目录路径
QDesktopServices::openUrl (QUrl::fromLocalFile(proj
ectDirectory)); //打开本地文件的函数
}

```

图 1

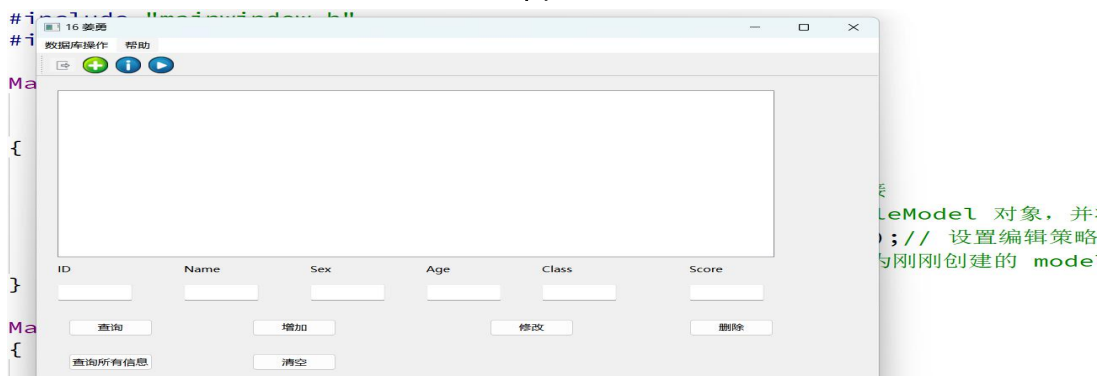


图 1 设计好的主窗口界面

图 2

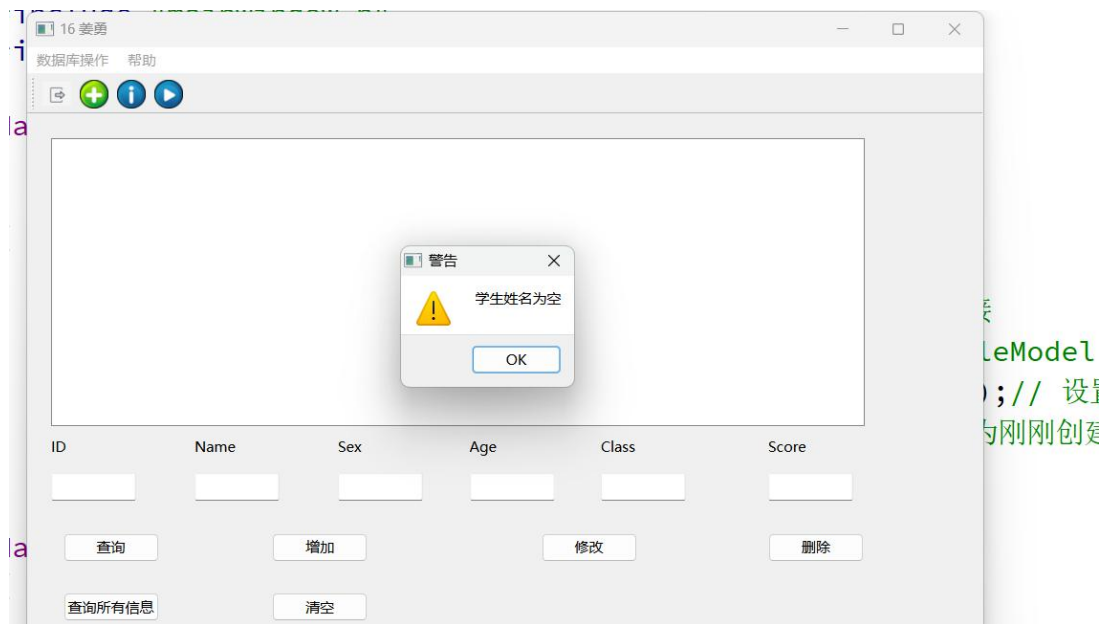


图 2 查询按钮学生姓名不能为空，不然弹出警告框

图 3

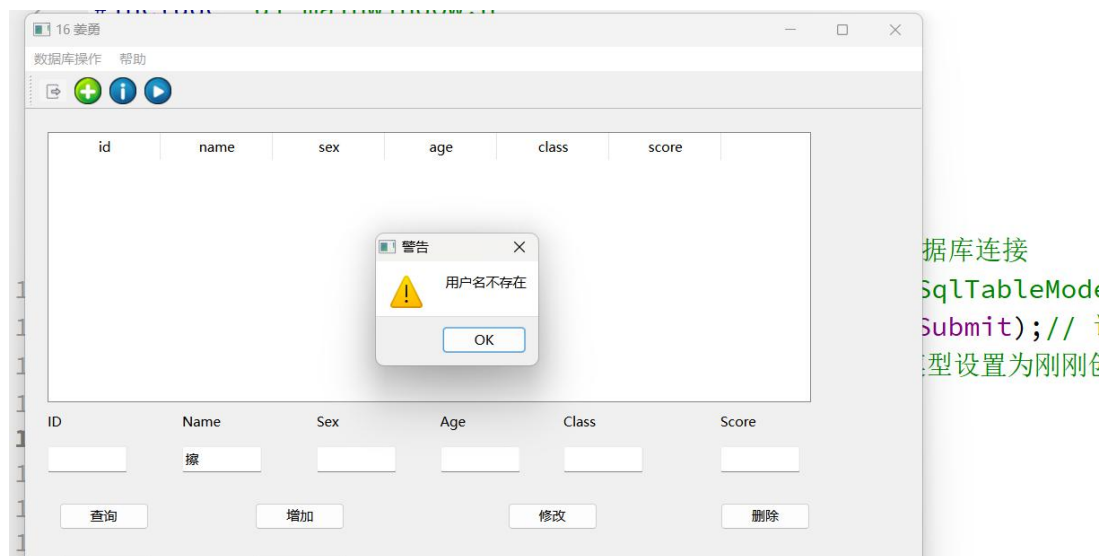


图 3 如果查询的用户不存在，则弹出警告框

图 4

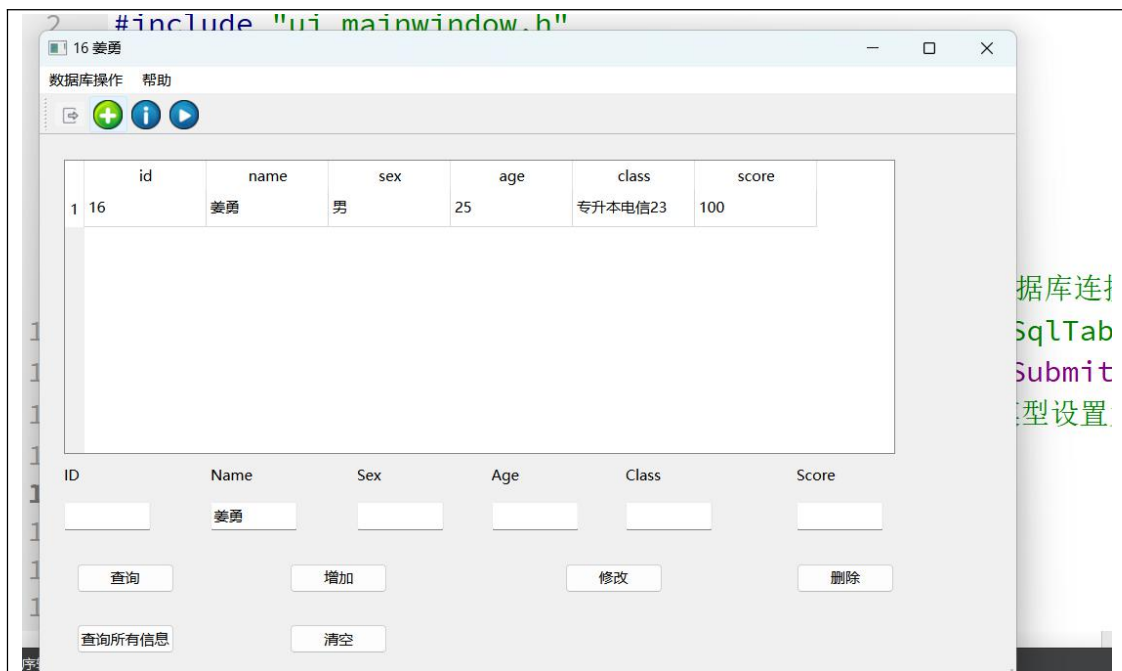


图 4 查询的用户存在，显示学生的详细信息

图 5

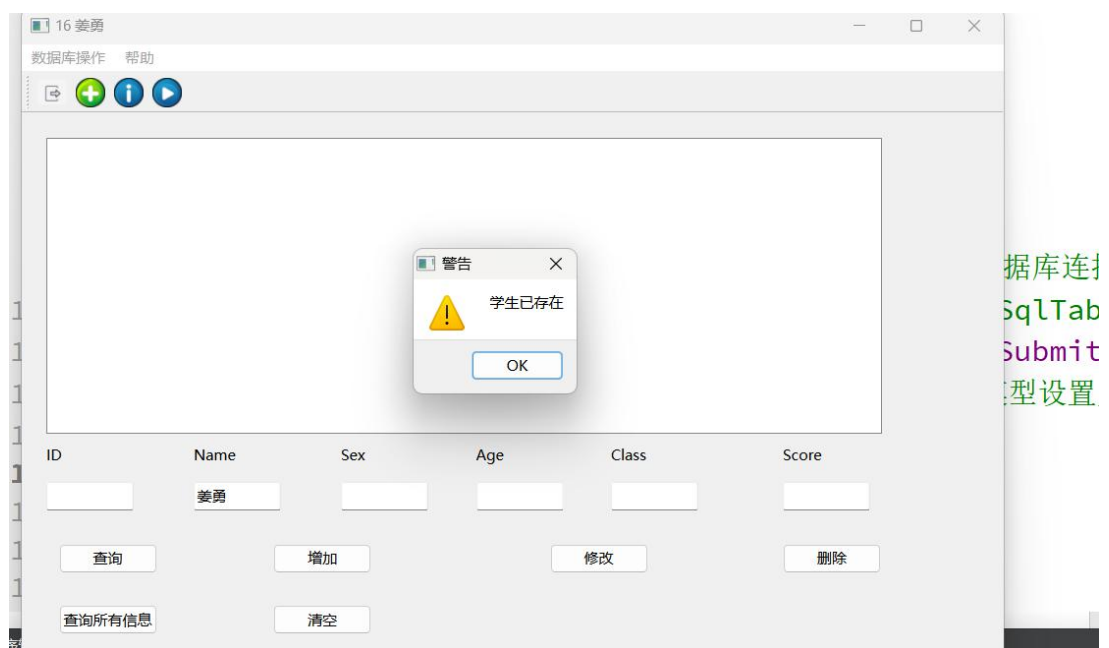


图 5 增加按钮点击时，必须输入完整的信息，不然弹出警告框，输入要增加的学生已存在，也弹出警告框

图 6

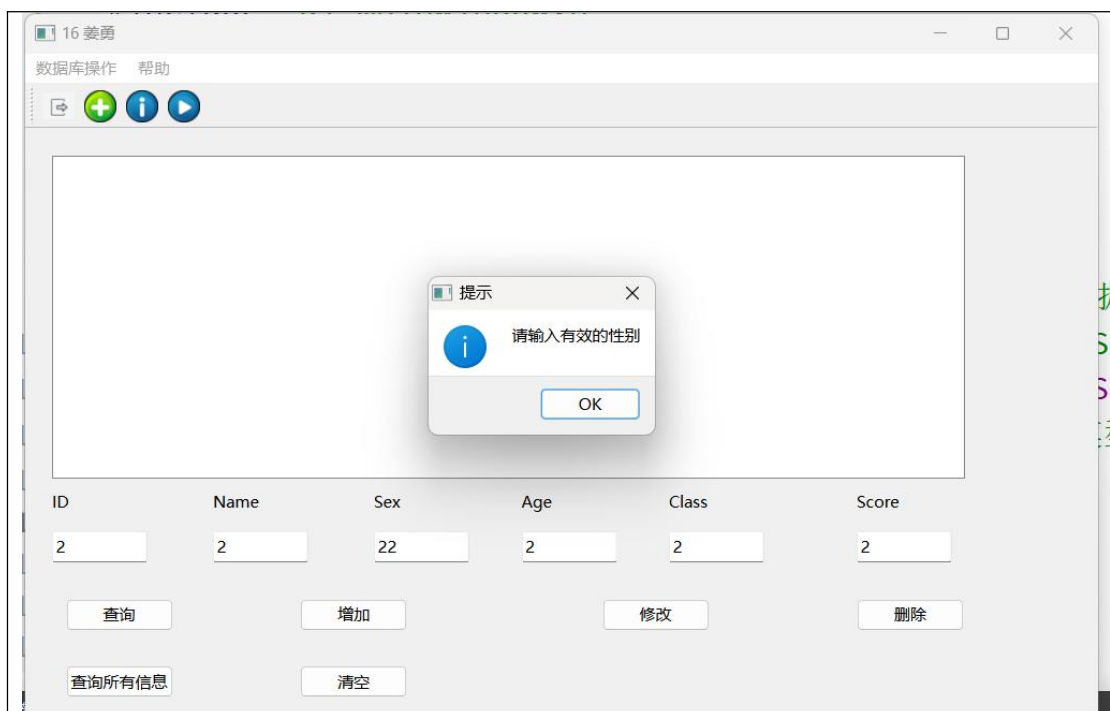


图 6 在增加学生的时候，给性别和年龄给了限制，必须符合要求

图 7

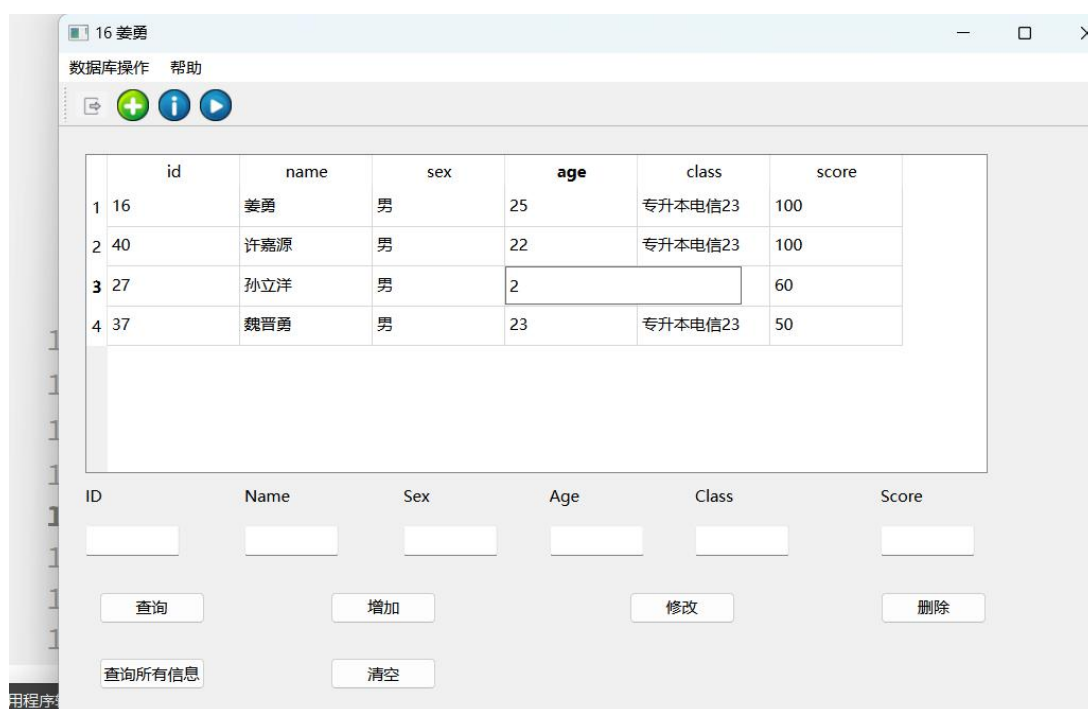


图 7 要修改信息的时候选择要修改的一项修改，点击修改按钮确认修改

图 8

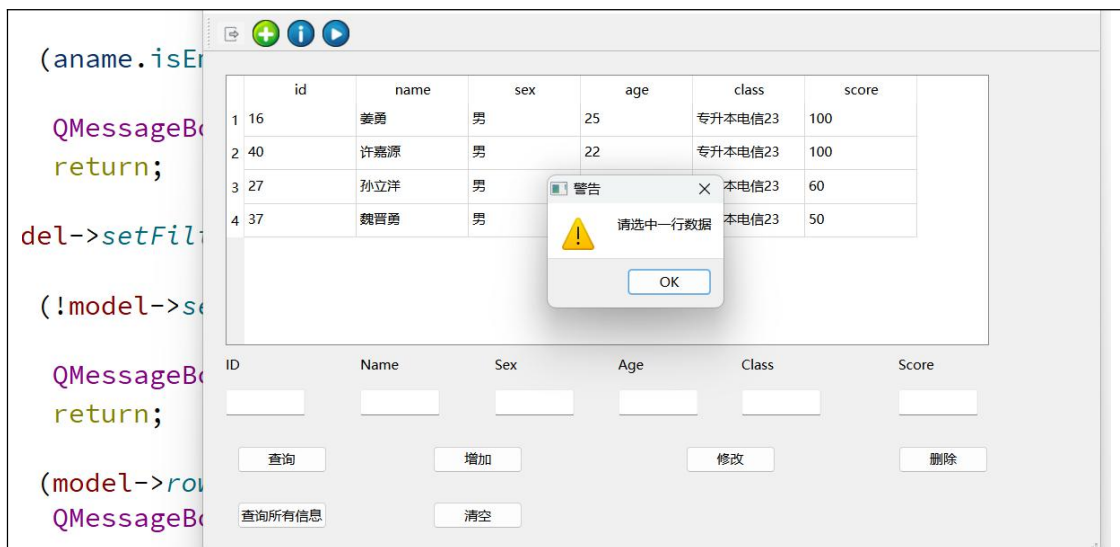


图 8 要删除一条数据的时候必须选择那一行数据，不然就会弹出警告框

图 9

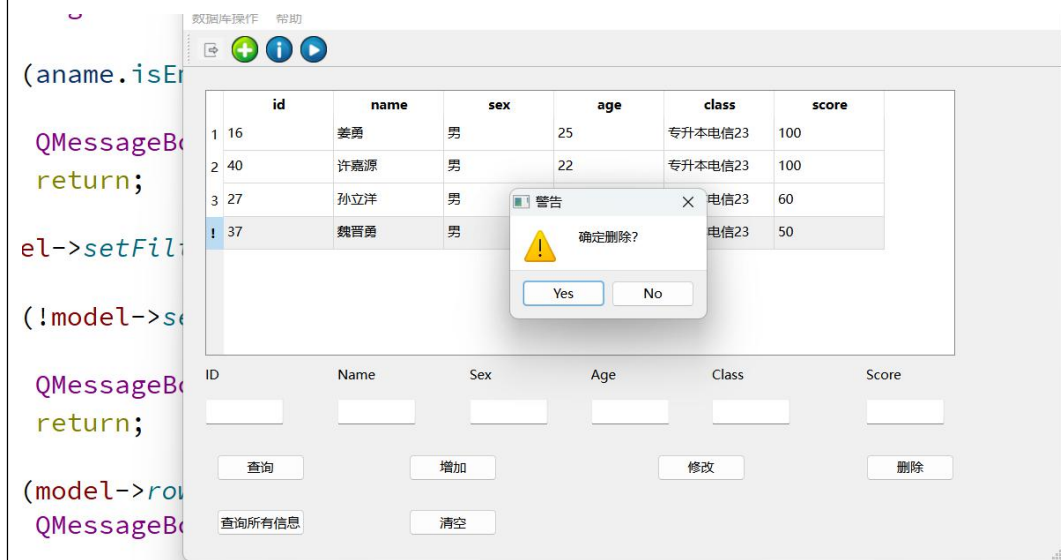


图 9 选中一行数据后点击删除按钮弹出警告框确认是否删除点击 yes 删除数据，点击 no 取消操作

图 10

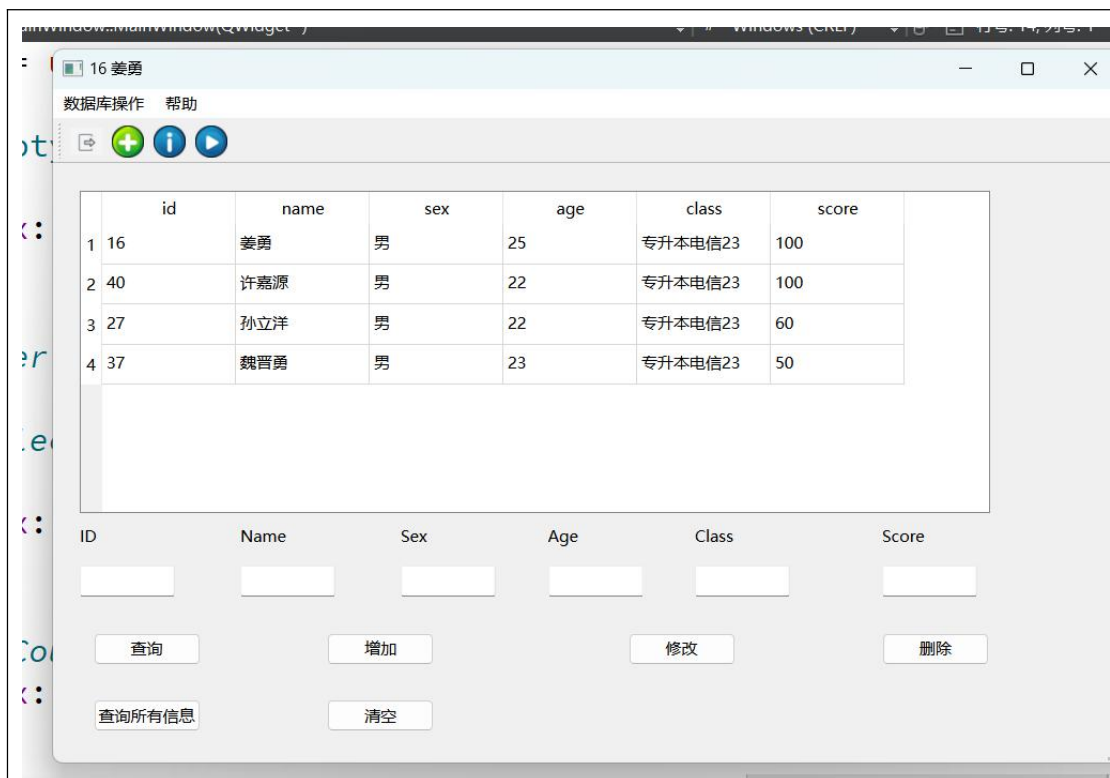


图 10 点击查询所有信息按钮显示数据库中所有的学生信息，清空按钮可以在增加查询出错的时候删除框中的值

图 11

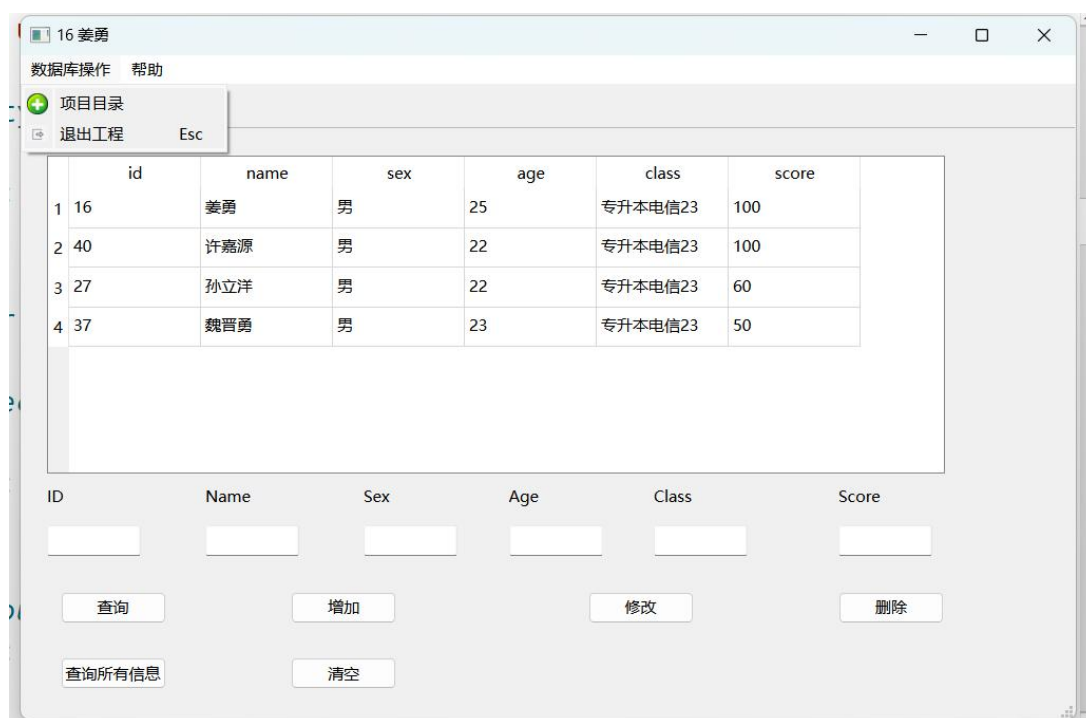


图 11 在菜单栏中给了 2 个菜单，在数据库操作菜单下有 2 个子项，选中项目目录可以打开本项目的本地存储地址，选项退出工程关闭页面

图 12

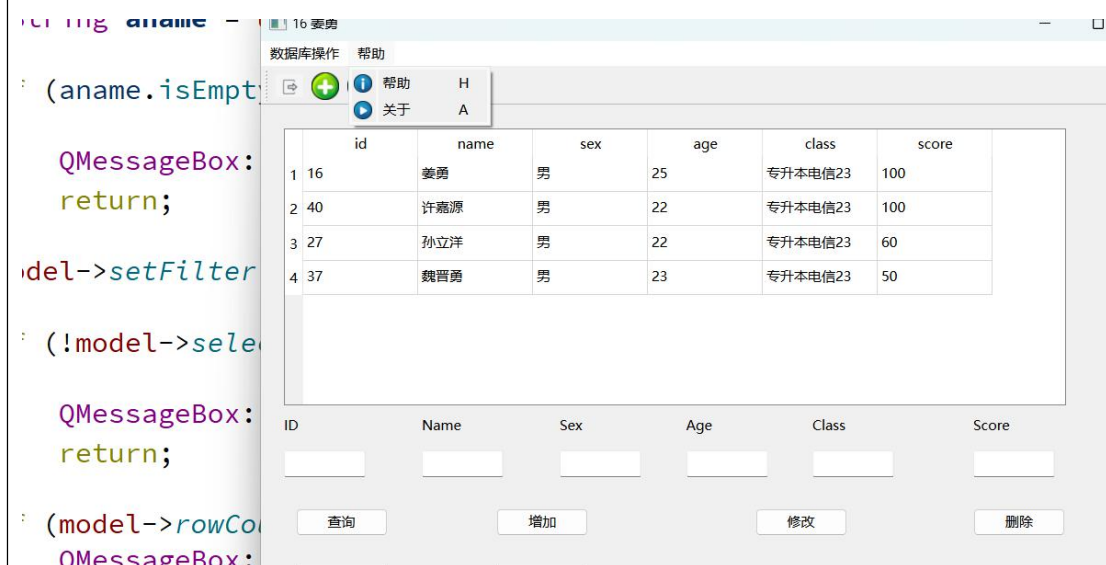


图 12 帮助菜单中也有 2 个子项，帮助中有软件的说明，关于中有作者的信息

四、实验总结

在本次实验中，成功地使用 Qt C++与 SQL Server 数据库进行了交互。通过配置环境、创建数据库连接、创建查询、执行查询、插入数据、更新数据和删除数据等步骤，完成了对数据库的基本操作。

在实验过程中，遇到了许多问题和挑战。例如，需要确保数据库驱动程序正确安装和配置，以建立与 SQL Server 的连接。此外，我们还遇到了查询错误和数据插入问题，需要仔细检查 SQL 语句和数据格式。

通过实验，深刻认识到了数据库操作的重要性。同时，我也学习到了如何使用 Qt C++进行数据库操作，以及如何解决可能出现的问题。

总之，本次实验让我们更加熟悉了 Qt C++与 SQL Server 数据库的交互操作，提高了我们的编程技能和实践能力。在未来的学习和工作中，我们将继续努力掌握更多关于数据库操作的知识和技术。

实验八成绩评定表

序号	考核项目	分值分布	成绩
1	出勤与纪律情况	10	
2	实验任务完成情况	40	
3	实验报告质量	50	
总分			
指导教师签字			